Course Title : HIGH PERFORMANCE COMPUTING Ass-6

Name : V.AksharaJyothi

Batch no : 12

Hall Ticket No : 2303A51793

1. **Lab**: NUMA Effects and Bandwidth Measurement

Objective:
To observe how memory placement and NUMA effects influence performance and to understand bandwidth limitations.

Task 1: System NUMA Information
Lscpu | grep -i numa

Observation:
Number of NUMA nodes = _____

```
nithish@LAPTOP-A2FOJ865:/mnt/c/Users/Sreeshma$ cd
nithish@LAPTOP-A2FOJ865:~$ mkdir numaass
nithish@LAPTOP-A2FOJ865:~$ cd numaass/
nithish@LAPTOP-A2FOJ865:~/numaass$ lscpu | grep -i numa
NUMA node(s):                         1
NUMA node0 CPU(s):                    0-7
nithish@LAPTOP-A2FOJ865:~/numaass$
```

2. **Task 2**: First Touch Memory Initialization

File: numa_first_touch.c

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;omp.h&gt;

#define N 10000000
```

06-02-2026 EOD

```
int main() {
double *A = (double*)malloc(sizeof(double)*N);

#pragma omp parallel for
for (int i = 0; i &lt; N; i++)
A[i] = 1.0;
```

double sum = 0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++)
sum += A[i];

printf("Sum = %f\n", sum);
free(A);
return 0;
}

Compile & Run:

gcc -O2 -fopenmp numa_first_touch.c -o numa_first_touch
./numa_first_touch

```
  GNU nano 7.2                                   numa_first_touch.c *
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 10000000

int main() {
    double *A = (double*)malloc(sizeof(double)*N);

    // First Touch Initialization
    #pragma omp parallel for
    for (int i = 0; i < N; i++)
        A[i] = 1.0;

    double sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; i++)
        sum += A[i];

    printf("Sum = %f\n", sum);

    free(A);
    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy       ^Q Where Was
```

```
nithish@LAPTOP-A2FOJ865:~/numaass$ nano numa_first_touch.c
nithish@LAPTOP-A2FOJ865:~/numaass$ gcc -O2 -fopenmp numa_first_touch.c -o numa_first_touch
nithish@LAPTOP-A2FOJ865:~/numaass$ NUMA node0 CPU(s): 0-7
-bash: syntax error near unexpected token `('
nithish@LAPTOP-A2FOJ865:~/numaass$ ./numa_first_touch
Sum = 10000000.000000
nithish@LAPTOP-A2FOJ865:~/numaass$
```

3. Task 3: **Conceptual Questions**

1. What is first- touch policy?

First-touch policy means that memory is physically assigned to the NUMA node of the thread that first uses (writes to) it.

When we use `malloc()`, memory is reserved but not actually mapped to a physical location immediately. The system waits until a thread writes to that memory. The NUMA node of that thread becomes the location where that memory page is placed.

In simple words:
The thread that first touches the memory decides where it lives.

This helps improve performance because threads can access memory that is closer to them.

2. Does NUMA affect correctness or performance?
NUMA does not affect correctness — your program will still give the correct output.

However, it does affect performance.

If a thread accesses memory that belongs to another NUMA node (remote memory), it takes more time compared to accessing its own local memory. This increases memory access delay and slows down the program.

So:

- Correctness → Not affected
- Performance → Can be affected significantly

3. Why is memory bandwidth important in NUMA systems?
Memory bandwidth refers to how fast data can move between memory and the CPU.

In NUMA systems, each node has its own local memory with good bandwidth. If a thread accesses its local memory, data transfer is fast. But if it accesses memory from another node, the bandwidth is lower and the access is slower.

When many threads try to access remote memory, the system can become slow because memory bandwidth becomes a bottleneck.

That's why proper memory placement (like first-touch) is important — it helps threads use local memory and improves overall performance